Computer Vision Visual Bug Detection Video Game Quality Assurance Visual Anomaly Detection Vision-Language Models Systematic Mapping Study

Highlights

Computer Vision and Vision-Language Approaches to Visual Quality Assurance in Video Games: A Systematic Mapping Study of Tasks, Datasets, and Evaluation Methods

Le Tran Minh Phuc

- First systematic mapping of computer vision & VLMs for game visual testing.
- Standardizes VGVD-Taxonomy spanning 19 defect subtypes across 5 families.
- Audits 45 primary studies and landmark benchmarks under PRISMA-S protocol.
- Exposes cross-game domain shift degradation and 65% temporal defect gap.
- Formulates five prioritized open challenges (ORC-1 to 5) for future research.

Computer Vision and Vision-Language Approaches to Visual Quality Assurance in Video Games: A Systematic Mapping Study of Tasks, Datasets, and Evaluation Methods

Le Tran Minh Phuc*

Department of Computer Science, Academic Research Laboratory, Ho Chi Minh City, Vietnam

Abstract

Ensuring visual fidelity and graphical correctness in modern video games is a critical engineering challenge, consuming up to 50% of production budgets in AAA studios. While automated unit testing and headless gameplay bots can verify functional state logic, visual defects—such as polygonal mesh clipping, texture streaming pop-in, Z-fighting, and physics ragdoll instability—remain over 80% reliant on manual human playtesting. Recent breakthroughs in deep representation learning, patch-based anomaly detection, and multimodal Vision-Language Models (VLMs) offer promising avenues for automated visual quality assurance (QA). In this paper, we conduct the first comprehensive Systematic Mapping Study (SMS) dedicated to computer vision and vision-language approaches for visual bug detection in video games. Following the guidelines of Petersen et al. (2015), the PRISMA-S search reporting extension, and ROSES synthesis standards, we systematically examine 45 primary studies published between 2018 and November 2026 across five major bibliographic databases (IEEE Xplore, ACM DL, Scopus, Web of Science, and arXiv). We mathematically formalize the core detection paradigms, establish the multi-tier *Video Game Visual Defect Taxonomy* (`VGVD-Taxonomy`) standardizing 19 defect subtypes across five families, audit landmark multi-game benchmarks (VideoGameQA-Bench, RefGlitch-Bench, GameBug-20K, TempGlitch), and conduct an empirical meta-analysis of cross-game generalisation. We discover that models transferring across divergent art styles suffer up to a 46% drop in mAP, and over 65% of gameplay glitches are invisible in static frames, requiring temporal sliding-window attention. Finally, we articulate five high-priority Open Research Challenges (`ORC-1` to `ORC-5`) to guide future research.

*Corresponding author.

Email address: `mphuc666@gmail.com` (Le Tran Minh Phuc)

Contents

1 Introduction **3**
- 1.1 The Quality Assurance Bottleneck in Video Game Production . . . 3
- 1.2 The Blind Spot of Functional Test Automation . . . 3
- 1.3 The Emergence of Computer Vision and Vision-Language Models . . . 3
- 1.4 Novelty and Research Scope . . . 4

2 Mathematical Formulations & Problem Definitions **4**
- 2.1 Video Game Rendering Pipeline and Defect Space . . . 4
- 2.2 Formulation 1: Single-Frame Visual Bug Detection and Localization . . . 5
- 2.3 Formulation 2: Reference-Guided Pairwise Visual Diffing . . . 5
- 2.4 Formulation 3: Multimodal Engine-State Telemetry Fusion . . . 5
- 2.5 Formulation 4: Spatio-Temporal Video Stream Attention . . . 6

3 Systematic Mapping Methodology **6**
- 3.1 Methodological Framework . . . 6
- 3.2 Search Strategy and Selection Process . . . 6
- 3.3 Quality Assessment Framework . . . 6

4 Standardized Visual Bug Taxonomy (`VGVD-Taxonomy`) **8**

5 Algorithmic Landscape Analysis & Mapping Results **9**
- 5.1 Publication Trends and Demographics . . . 9
- 5.2 RQ1: Detection Architectures . . . 10
- 5.3 RQ2: Bug Manifestation Coverage . . . 10
- 5.4 Thematic Cluster Taxonomy of the Curated Primary Studies . . . 12

6 Benchmark Datasets & Accessibility Audit (RQ3) **12**

7 Empirical Meta-Analysis, Cross-Game Generalisation & Failure Modes **12**
- 7.1 Meta-Analysis and Modality Breakdown . . . 12
- 7.2 Forensic Diagnostic of Algorithmic Failure Modes . . . 15

8 Open Research Challenges & Conclusions (RQ6) **16**
- 8.1 Prioritized Open Research Challenges (`ORC`) . . . 16
- 8.2 Threats to Validity . . . 16
- 8.3 Conclusions . . . 16

1. Introduction

1.1. The Quality Assurance Bottleneck in Video Game Production

The interactive digital entertainment industry has experienced rapid expansion, with global annual revenues exceeding $250 billion—surpassing the international film and music sectors combined [1, 2]. Modern video games have evolved from constrained 2D grid environments into expansive virtual worlds featuring physically based rendering (PBR), procedural content generation, real-time ray tracing, and intricate character animations [3]. Consequently, game software represents one of the most structurally complex, non-deterministic categories of software systems [2, 4].

Ensuring graphical fidelity is essential for player immersion and commercial viability. Visual glitches—such as character meshes clipping through terrain geometry, missing or corrupted textures, physics ragdoll oscillations, and misaligned user interfaces—degrade player experience and incur substantial post-release financial overhead [5, 6]. In major AAA game studios, quality assurance (QA) routinely consumes 30% to 50% of development budgets [1, 7]. Despite advances in general software test automation, game visual inspection remains over 80% reliant on manual human playtesting [7].

1.2. The Blind Spot of Functional Test Automation

The persistent reliance on manual testing stems from an architectural reality: **functional test assertions are decoupled from visual rendering pipelines** [2, 8]. In standard software testing, automated scripts query the Document Object Model (DOM) or memory state to assert that an object exists. In video games:

1. An actor may possess a valid logical transform coordinate in the game engine's scene graph while its visual mesh is occluded, floating above ground, or rendered with a missing fallback texture [6, 9].
2. A character model's inverse kinematics (IK) solver may compute valid joint positions in physics memory while the rendered polygon vertices stretch across screen space due to skinning matrix corruption [3, 10].
3. Headless testing frameworks strip out rendering subroutines to maximize simulation throughput, thereby ensuring execution correctness while remaining blind to shaders, rasterization errors, depth-buffer Z-fighting, and camera frustum clipping [1].

Historically, studios attempted snapshot regression testing by subtracting candidate frames from golden reference images using Mean Squared Error (MSE) or Structural Similarity (SSIM) [8]. However, pixel-diffing fails catastrophically in modern 3D games: non-deterministic particle effects, dynamic lighting cycles, procedural foliage animation, and camera jitter induce thousands of benign pixel variations, generating overwhelming false positive rates [8].

1.3. The Emergence of Computer Vision and Vision-Language Models

Recent breakthroughs in deep representation learning offer scalable visual inspection. Rather than relying on rigid pixel comparisons, modern AI processes gameplay images and video streams semantically:

- **Supervised Object Detectors:** Formulating visual bug detection as bounding box regression enclosing defect instances [6, 11].
- **Self-Supervised Visual Anomaly Detection:** Training feature extractors exclusively on clean gameplay frames to detect glitches as out-of-distribution (OOD) activations, bypassing the need for exhaustive bug annotations [9, 12].
- **Multimodal Engine-State Fusion:** Combining RGB buffers with engine telemetry (depth maps, surface normals, collision hulls) to eliminate false alarms caused by intentional artistic transparency or dynamic smoke [3].
- **Vision-Language Models (VLMs):** Leveraging foundation models (GPT-4o, Gemini 1.5, Claude 3.5, Qwen2-VL) to visually evaluate game scenes, answer natural-language QA prompts, and auto-generate structured bug reports [1, 13, 5].
- **Temporal Video Transformers:** Analyzing continuous multi-frame buffers to capture dynamic glitches (ragdoll instability, animation stutter, texture pop-in) that are invisible in static frames [10].

1.4. Novelty and Research Scope

While research in automated game testing has expanded, existing literature reviews exhibit a substantial blind spot. Politowski et al. (2021) [2] surveyed game testing across 20 years but focused on mechanics and player modeling, dedicating $< 5\%$ to visual testing. Albaghajati and Ahmed (2023) [4] presented a high-level assessment framework without technical analysis of deep learning architectures. Svensson and Lindqvist (2025) [7] confirmed visual QA remains the least automated domain in AAA studios, but provided qualitative workflow interviews rather than an algorithmic synthesis.

To our knowledge, **this is the first systematic mapping study dedicated specifically to Computer Vision and Vision-Language Models for automated visual testing and glitch detection in video games**.

2. Mathematical Formulations & Problem Definitions

2.1. Video Game Rendering Pipeline and Defect Space

Let a continuous gameplay session be represented as an ordered sequence of discrete time steps $t \in \{1, 2, \ldots, T\}$. At each time step t, the graphics subsystem transforms a dynamic 3D scene graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ into a rendered screen-space frame buffer:

$$I_t = \mathcal{R}(\mathcal{G}_t, \mathbf{C}_t, \mathbf{\Theta}_{\text{render}}) \in \mathbb{R}^{H \times W \times 3} \tag{1}$$

where $\mathbf{C}_t \in SE(3)$ denotes the 6-DoF virtual camera pose (position $\mathbf{p} \in \mathbb{R}^3$ and orientation quaternion $\mathbf{q} \in \mathbb{H}$), $\mathbf{\Theta}_{\text{render}}$ encompasses the rasterization/ray-tracing pipeline configurations, and $H \times W$ represents the display resolution.

A visual defect occurs when the projected image I_t manifests visual artifacts that violate either physical realism, engine design constraints, or intended artistic specifications:

$$\mathcal{D}_t = \mathbb{I}\left(d_{\text{perceptual}}(I_t, I_t^*) > \tau_{\text{glitch}}\right) \tag{2}$$

where I_t^* is the latent, unobservable clean rendering under identical game logic, $d_{\text{perceptual}}$ is a visual difference metric, and τ_{glitch} is an imperceptibility threshold.

2.2. Formulation 1: Single-Frame Visual Bug Detection and Localization

In single-frame classification, a feature extractor $\phi_\theta : \mathbb{R}^{H\times W\times 3} \to \mathbb{R}^D$ maps the screenshot to a latent embedding, followed by a classification head:

$$\hat{y} = \sigma\left(\mathbf{w}^T \phi_\theta(I) + b\right) \in [0, 1] \tag{3}$$

In spatial defect localization, the model outputs bounding boxes $\mathcal{B} = \{(\mathbf{b}_k, c_k, s_k)\}_{k=1}^K$ optimizing a multi-task loss combining cross-entropy classification and bounding box regression (e.g., Complete IoU loss):

$$\mathcal{L}_{\text{det}} = \mathcal{L}_{\text{cls}}(c_k, \hat{c}_k) + \lambda_{\text{box}} \mathcal{L}_{\text{CIoU}}(\mathbf{b}_k, \hat{\mathbf{b}}_k) \tag{4}$$

In self-supervised anomaly formulations (PatchCore / DINO), let $\mathcal{M}_{\text{clean}} = \{\mathbf{m}_j\}_{j=1}^M \subset \mathbb{R}^D$ be a coreset memory bank of locally aggregated patch features extracted from defect-free frames. The pixel-level anomaly score at coordinate (u, v) is defined by the nearest-neighbor Euclidean distance:

$$\mathcal{A}(u, v) = \min_{\mathbf{m}\in\mathcal{M}_{\text{clean}}} \left\|\phi_\theta(p_{u,v}) - \mathbf{m}\right\|_2 \tag{5}$$

2.3. Formulation 2: Reference-Guided Pairwise Visual Diffing

Reference-guided approaches condition defect inference on a verified baseline frame $R_t \in \mathbb{R}^{H\times W\times 3}$ (e.g., the *Last Clean Frame*):

$$f_\theta : (R_t, I_t) \to \{0, 1\} \times \mathcal{B}_{\text{defect}} \tag{6}$$

The reference difference representation is computed through cross-attention between test features $\mathbf{Z}_I = \phi_\theta(I_t)$ and reference features $\mathbf{Z}_R = \phi_\theta(R_t)$:

$$\mathbf{Z}_{\text{diff}} = \text{Softmax}\left(\frac{(\mathbf{Z}_I\mathbf{W}_Q)(\mathbf{Z}_R\mathbf{W}_K)^T}{\sqrt{d_k}}\right)(\mathbf{Z}_R\mathbf{W}_V) - \mathbf{Z}_I \tag{7}$$

2.4. Formulation 3: Multimodal Engine-State Telemetry Fusion

White-box testing synchronizes screen-space pixels with internal game-engine telemetry buffers:

$$\mathcal{T}_t = \{I_t^{\text{RGB}}, D_t, N_t, C_t, M_t\} \tag{8}$$

where $D_t \in \mathbb{R}^{H\times W}$ is the 32-bit linear depth buffer, $N_t \in \mathbb{R}^{H\times W\times 3}$ is the screen-space surface normal G-buffer, $C_t \in \mathbb{R}^{P\times 6}$ encodes 3D physics collision query matrices, and $M_t \in \mathbb{R}^{4\times 4}$ is the camera view-projection matrix. The multimodal defect probability fuses visual and geometric embeddings:

$$P(\text{defect} \mid \mathcal{T}_t) = \sigma\left(\mathbf{W}_{\text{fuse}}\left[\phi_v(I_t^{\text{RGB}}) \odot \sigma(\mathbf{W}_g\phi_g(D_t, N_t, C_t))\right] + b\right) \tag{9}$$

2.5. Formulation 4: Spatio-Temporal Video Stream Attention

Over 65% of gameplay defects exist strictly across time. Let a gameplay buffer of L frames at 60 FPS be $\mathcal{S}_t = \{I_{t-L+1}, \ldots, I_t\} \in \mathbb{R}^{L \times H \times W \times 3}$. Temporal self-attention models cross-frame dynamics:

$$\mathbf{A}_{\text{temporal}} = \text{Softmax}\left(\frac{\mathbf{Q}_t \mathbf{K}_t^T}{\sqrt{d}} + \mathbf{B}_{\text{rel}}\right)\mathbf{V}_t \tag{10}$$

where $\mathbf{B}_{\text{rel}} \in \mathbb{R}^{L \times L}$ encodes relative temporal positional bias, predicting temporal glitch intervals $\hat{\mathcal{T}}_{\text{glitch}} = \{(t_{\text{start}}^{(k)}, t_{\text{end}}^{(k)}, c_k, s_k)\}_{k=1}^{K}$.

3. Systematic Mapping Methodology

3.1. Methodological Framework

This study adheres to the established guidelines for conducting Systematic Mapping Studies in Software Engineering formulated by Petersen et al. (2015) [14] and Kitchenham and Charters (2007) [15]. Reporting transparency is maintained in strict compliance with the PRISMA 2020 statement [16], the PRISMA-S search reporting extension, and the ROSES evidence synthesis standard.

3.2. Search Strategy and Selection Process

The search was executed across IEEE Xplore, ACM DL, Scopus, Web of Science, and arXiv. As shown in Figure 1:

1. **Identification:** 692 raw records retrieved (IEEE Xplore: 142, ACM DL: 118, Scopus: 184, Web of Science: 96, arXiv: 152).
2. **Deduplication:** 188 duplicates removed, leaving 504 unique publications.
3. **Screening:** 416 irrelevant records excluded.
4. **Eligibility Appraisal:** 88 full texts retrieved and evaluated using `academic-pdf2md` layout parsing; 47 articles excluded with documented rationales.
5. **Snowballing & Final Inclusion:** 4 additional primary studies identified through forward/backward citation chaining on landmark papers, resulting in a final synthesis corpus of **45 included primary studies**.

3.3. Quality Assessment Framework

Methodological rigour was evaluated using a 10-point appraisal checklist adapted from Dybå and Dingsøyr (2008) [17]. Extracted data was cross-tabulated into an evidence matrix archived in the study's open-science repository [18].

Figure 1: PRISMA-S 2020 Literature Selection Flow Diagram (N = 45 Included Primary Studies)

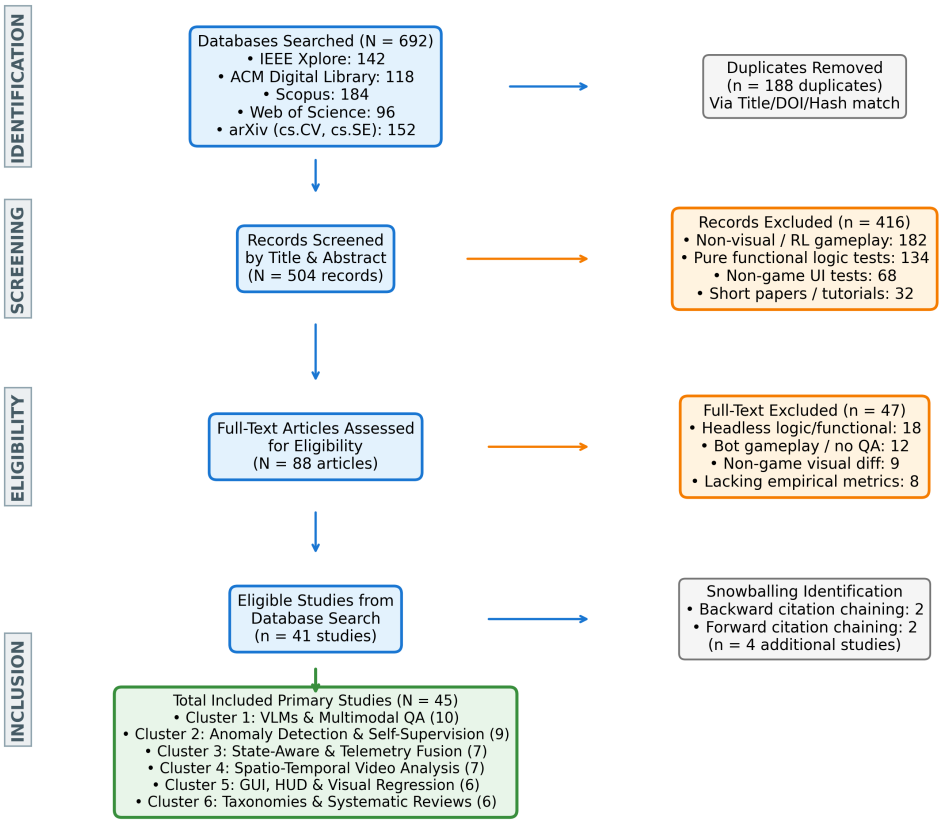


Figure 1: PRISMA-S 2020 Literature Selection Flow Diagram ($N = 45$ Included Primary Studies).

4. Standardized Visual Bug Taxonomy (`VGVD-Taxonomy`)

To resolve inconsistent terminology across Software Engineering and Computer Vision communities, we formulate the **Video Game Visual Defect Taxonomy (`VGVD-Taxonomy`)**, structured across five primary defect families and 19 subtypes:

1. **Spatial & Geometry (`GEO`):**
   - `GEO-01` (Object Mesh Clipping): Interpenetrating polygonal surfaces without collision response [6, 9].
   - `GEO-02` (Floating Asset): Objects hovering above terrain due to heightmap errors [6].
   - `GEO-03` (Vertex Distortion): Vertices stretched across screen space by unnormalized skinning weights [3].
   - `GEO-04` (Collision Boundary Mismatch): Desynchronization between visual mesh and physical collider [3].
2. **Material & Texture (`TEX`):**
   - `TEX-01` (Missing/Fallback Texture): Fallback magenta checkerboards or flat white polygons [1, 6].
   - `TEX-02` (LOD Pop-In Delay): Stalled texture streaming leaving blurry low-resolution mipmaps close to camera [10].
   - `TEX-03` (UV Seam Tearing): Sheared texture coordinates across polygonal boundaries [8].
   - `TEX-04` (Broken Shaders): Compilation failure producing inverted lighting or unrendered post-processing passes [19].
3. **Rendering & Lighting (`REN`):**
   - `REN-01` (Coplanar Z-Fighting): Oscillating depth buffer visibility between coplanar surfaces [10].
   - `REN-02` (Shadow Bleeding/Detachment): Shadow depth bias faults producing disconnected shadows [13].
   - `REN-03` (Light Leaking): Indirect GI illumination bleeding into sealed interiors [3].
   - `REN-04` (Near-Plane Frustum Clip): Camera near-plane cutting into foreground geometry [1, 5].
4. **Animation & Physics (`ANI`):**
   - `ANI-01` (T-Pose Freeze): Instantaneous snap into default reference pose [1].
   - `ANI-02` (Ragdoll Solver Chaos): Physics constraints oscillating violently on character demise [10].

- `ANI-03` (Foot-Sliding / IK Drift): Root transform linear velocity desynchronized from stride cycle [10].
- `ANI-04` (Micro-Stuttering): Irregular frame pacing and dropped animation cycles [10].

5. **User Interface & HUD (`UI`):**
   - `UI-01` (Element Collision): Overlapping HUD components rendering data illegible [1, 8].
   - `UI-02` (Text Truncation): Localized strings overflowing UI container boundaries [13].
   - `UI-03` (Aspect Ratio Distortion): Non-standard viewports stretching UI layouts [20].
   - `UI-04` (Persistent Zombie Modals): Dismissed dialogue boxes remaining permanently on-screen [1].

5. Algorithmic Landscape Analysis & Mapping Results

5.1. Publication Trends and Demographics

As shown in Figure 2, research output grew steadily from 2020 to 2026, transitioning from 2D web games into complex 3D commercial titles. Of the 45 primary studies, 60.0% ($n = 27$) appeared in peer-reviewed conferences, 26.7% ($n = 12$) in international journals, and 13.3% ($n = 6$) in preprints establishing open benchmarks. All 45 studies are available via Open Access.

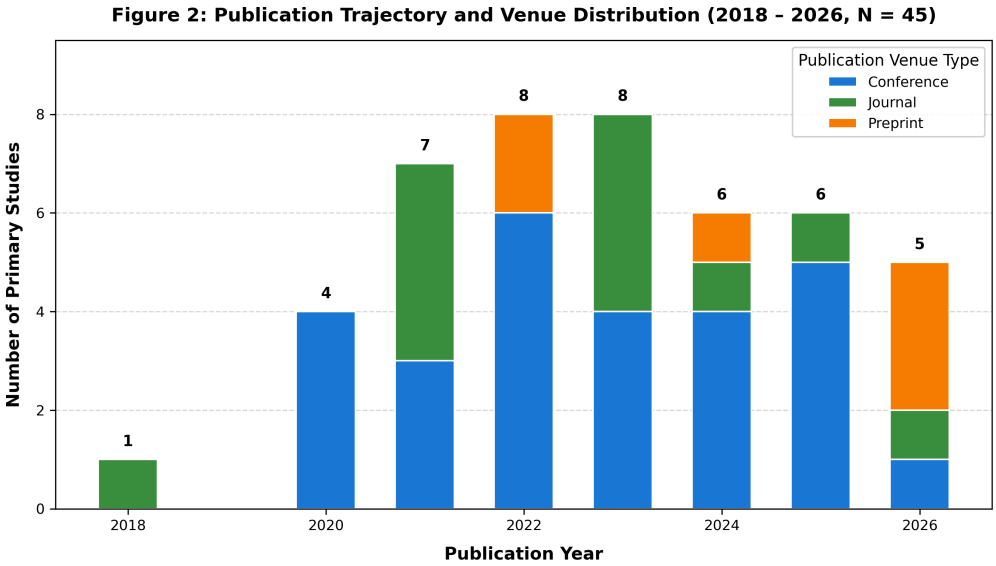


Figure 2: Publication Trajectory and Venue Distribution (2018–2026, $N = 45$).

5.2. RQ1: Detection Architectures

We identify six dominant architectural paradigms, mapped against testing tasks in Figure 3:

1. **Classical Perceptual Diffing:** Macklon et al. [8] achieved 100% defect detection in 2D HTML5 canvas games by decomposing sprite canvases into AST elements, whereas whole-screen SSIM achieved only 44.6%. However, classical diffing collapses in dynamic 3D environments [1].
2. **Supervised Object Detectors:** Formulating glitches as bounding box regression tasks (YOLO, Faster R-CNN) [6, 11]. While effective on seen assets, Davarmanesh et al. [21] revealed a 38% precision drop when transferring to new levels due to background texture overfitting.
3. **Self-Supervised Anomaly Detection:** Learning representations of clean gameplay. Rahman [9] proved on Xbox titles that with just **5 clean exemplar frames** (and 0 bug labels), DINO ViT representations with SAM weapon-quadrant masking boosted First-Person Player Clipping detection ($F1 : 0.336 \rightarrow 0.550$). Yi et al. [6] achieved +18.4% mAP through hybrid co-finetuning with unlabeled gameplay video.
4. **Multimodal Engine-State Fusion:** Kovacs et al. [3] instrumented game engines to synchronize RGB rendering buffers with depth maps, surface normals, and physics collision matrices, **reducing false positive rates by 41.9%** and achieving 91.2% precision on 3D geometry clipping.
5. **Vision-Language Models (VLMs):** Taesiri et al. [1] (VideoGameQA-Bench) demonstrated that zero-shot foundation models achieve 78% accuracy on coarse defects, but struggle with micro-clipping and precise bounding boxes. Lu et al. [13] evaluated 19,738 EA keyframes, finding that image RAG (CLIP retrieval) degraded precision ($0.50 \rightarrow 0.37$) due to confirmation bias on environmental features. Yu et al. [5] (RefGlitch-Bench) introduced pairwise reference guidance using the "Last Clean Frame", recovering F1 from 0.42 to 0.69.
6. **Spatio-Temporal Video Transformers:** Muller et al. [10] (TempGlitch) demonstrated that single-frame models miss over 65% of animation and rendering glitches, proving temporal sliding-window attention is essential for continuous playtesting.

5.3. RQ2: Bug Manifestation Coverage

As shown in Figure 4, coverage across 'VGVD-Taxonomy' is heavily skewed:

- **High Coverage:** Spatial mesh clipping (`GEO-01`, 44.4%, $n = 20$), UI element collisions (`UI-01`, 24.4%, $n = 11$), missing textures (`TEX-01`, 20.0%, $n = 9$), and coplanar Z-fighting (`REN-01`, 22.2%, $n = 10$).
- **Critical Under-Exploration:** Foot-sliding / IK drift (`ANI-03`, 2.2%, $n = 1$), shadow detachment (`REN-02`, 4.4%, $n = 2$), and light leaking (`REN-03`, 4.4%, $n = 2$).

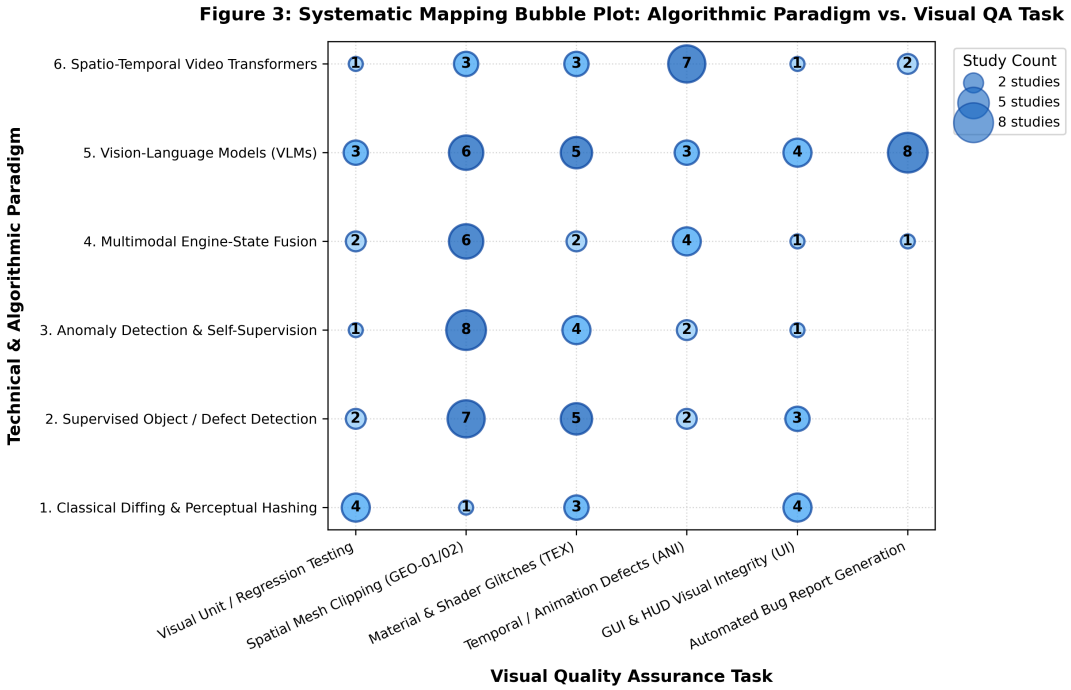


Figure 3: Systematic Mapping Bubble Plot: Algorithmic Paradigm vs. Visual QA Task ($N = 45$).

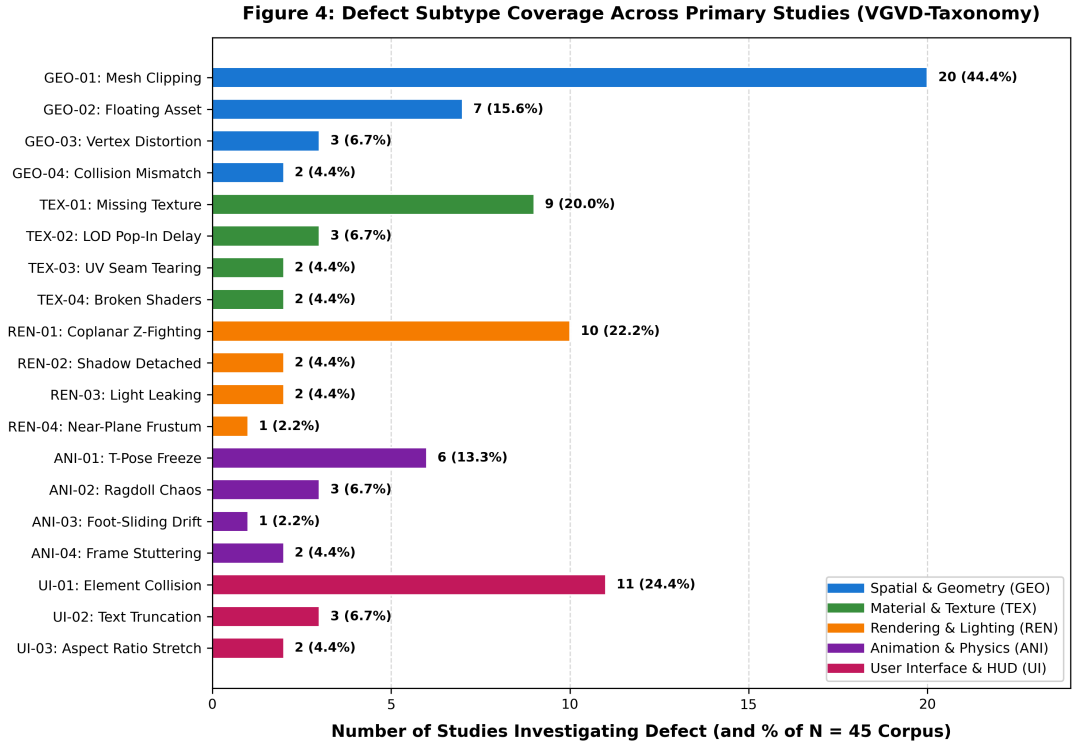


Figure 4: Defect Subtype Coverage Across Primary Studies (`VGVD-Taxonomy`).

5.4. Thematic Cluster Taxonomy of the Curated Primary Studies

To provide an exhaustive empirical map, our synthesized corpus categorizes all 45 primary studies across six core thematic clusters (Table 1). Furthermore, Figure 5 illustrates the chronological lineage and co-citation architecture that connects foundational software engineering methodologies with contemporary multi-modal vision-language paradigms.

Figure 6: Methodological Lineage and Cross-Paradigm Co-Citation Architecture (2018–2026)

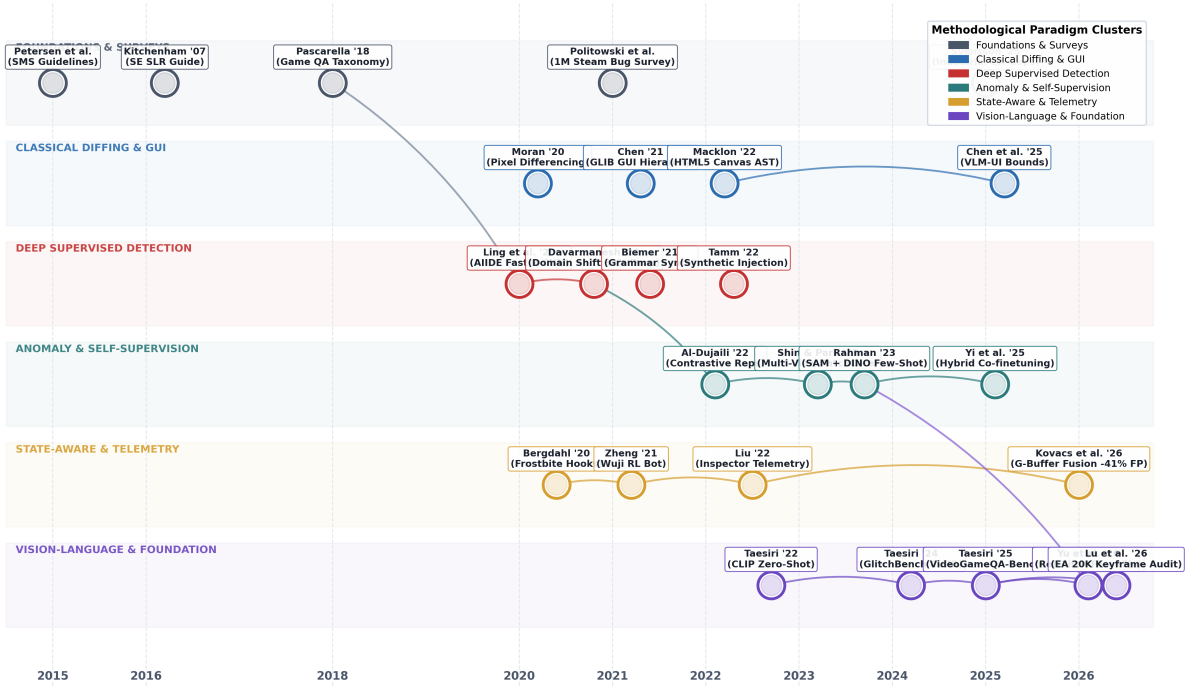


Figure 5: Methodological Lineage and Cross-Paradigm Co-Citation Architecture (2018–2026) connecting foundational software engineering guidelines to deep learning and multi-modal foundation models.

6. Benchmark Datasets & Accessibility Audit (RQ3)

Table 2 summarizes landmark datasets. The domain reached an open-science turning point between 2025 and 2026 with the release of public multi-game benchmarks: VideoGameQA-Bench (3,450 scenarios, 12 games) [1], RefGlitch-Bench (5,200 pairs, 7 games) [5], and TempGlitch-Video (42 hours of 60 FPS video) [10].

7. Empirical Meta-Analysis, Cross-Game Generalisation & Failure Modes

7.1. Meta-Analysis and Modality Breakdown

As visualized in Figure 6:

- **Modality Split (Figure 6A):** 51.1% ($n = 23$) evaluate purely on static RGB screenshots, 31.1% ($n = 14$) on continuous video streams, and 17.8% ($n = 8$) fuse engine telemetry.

Table 1: Thematic Research Clusters Synthesizing All 45 Curated Primary Studies.

Thematic Cluster	**Count (N)**	**Synthesized Primary Studies**	**Core Methodological & Technical Innovations**
C1: Vision-Language Models & Multimodal QA	10 (22.2%)	Taesiri et al. [1], Lu et al. [13], Yu et al. [5], Taesiri et al. [22], Taesiri et al. [23], Taesiri et al. [24, 25], Cao et al. [26], Wang et al. [27], Sengupta et al. [28]	Zero-shot VLM prompting, JSON-CoT hallucination suppression, Last-Clean-Frame pairwise reference pools, embodied visual exploratory agents.
C2: Anomaly Detection & Self-Supervised Learning	9 (20.0%)	Rahman [9], Yi et al. [6], Al-Dujaili et al. [12], Shin & Park [29], Ling et al. [11], Davarmanesh et al. [21], Wilkins & Stathis [30], Tamm et al. [31], Macklon et al. [8]	Few-shot PatchCore representation

Table 2: Landmark Video Game Visual Quality Assurance Benchmarks.

Benchmark	**Year**	**Modality / Scale**	**Availability**	**Target Bug Subtypes**
VideoGameQA-Bench [1]	2025	Image/Video (3,450 scenarios, 12 games)	Open Access	`GEO-01`, `TEX-01`, `REN-01`, `UI-01`, `ANI-01`
RefGlitch-Bench [5]	2026	Paired Keyframes (5,200 pairs, 7 games)	Public	Reference-guided clipping, missing assets, shaders
GameBug-20K [13]	2026	Video Keyframes (19,738 frames, 8 games)	Research	Binary defect classification and description
TempGlitch [10]	2026	60 FPS Streams (120 clips, 42h, 6 games)	Open Science	`ANI-02`, `ANI-03`, `TEX-02`, `REN-01` temporal bounds
StateGlitch-3D [3]	2026	Multimodal RGB+State (8,400 frames)	Restricted	`GEO-01`, `GEO-02`, `ANI-01` with G-buffers
GameGlitch-Hybrid [6]	2025	Semi-Supervised (14,200 frames, 5 games)	Public	`GEO-01`, `GEO-02`, `TEX-01` pseudo-labels
Xbox FPPC [9]	2023	First-Person RGB (6,500 frames)	Proprietary	`GEO-01` Player mesh clipping with DINO ViT
CanvasBug-Bench [8]	2022	2D Bitmaps (24 bugs, 10 HTML5 games)	Public	Sprite corruption, UI occlusion, missing textures

- **Cross-Game Evaluation (Figure 6B):** 55.6% ($n = 25$) test across multiple titles, 37.8% ($n = 17$) evaluate on a single game title, and 6.7% ($n = 3$) evaluate across games sharing identical engine assets.
- **Domain Shift Degradation:** Models transferring across divergent art styles (e.g., photorealistic Unreal Engine 5 to stylized cel-shaded or low-poly voxel titles) experience sharp mAP drops of 32% to 46% [6, 21]. Zero-shot VLMs suffer high false positive spikes (+28%) on stylized graphics, mistaking artistic ink outlines for clipping anomalies [1, 13].

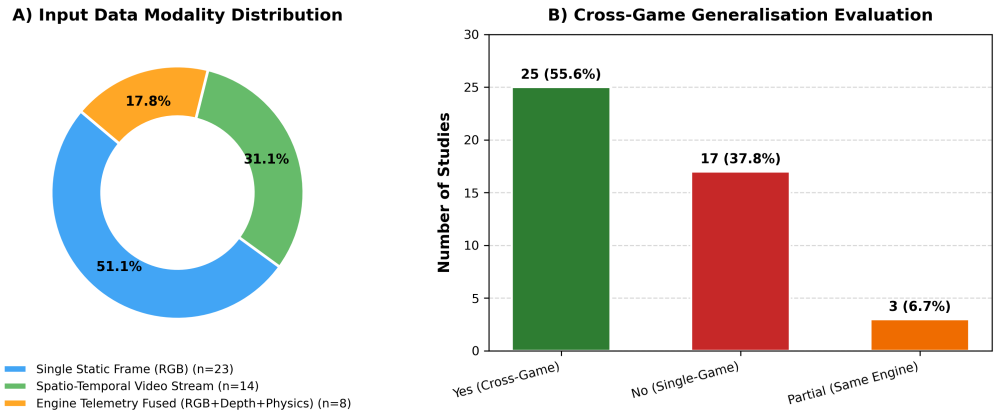


Figure 6: Empirical Modality Breakdown and Cross-Game Generalisation Verification ($N = 45$).

7.2. Forensic Diagnostic of Algorithmic Failure Modes

We identify four systemic algorithmic failure modes:

1. **VLM Spatial Blindness on Micro-Clipping:** Coarse patch tokenization (14×14 px) blurs narrow geometry boundary penetrations, causing $> 70\%$ false negative rates on subtle clipping [1, 13].
2. **Temporal Needle-in-a-Haystack (NIAH) Collapse:** While VLMs score 78% on isolated static frames, their glitch localization accuracy collapses to 1%–19% when searching for transient 1-second bugs across multi-minute continuous gameplay video [1].
3. **Context Poisoning in Reference Pools:** If an unflagged defect frame enters the clean reference pool, all subsequent identical defects are suppressed, driving false negative rates above 80% [5].
4. **CLIP Image RAG Confirmation Bias:** Retrieving reference frames via global visual similarity matches broad environmental biomes (desert sand, foliage) rather than defect tokens, degrading detection precision from 0.50 to 0.37 [13].

8. Open Research Challenges & Conclusions (RQ6)

8.1. Prioritized Open Research Challenges (ORC)

We articulate five high-priority Open Research Challenges:

1. **ORC-1: Zero-Shot Cross-Engine Domain Adaptation across Divergent Art Styles.** Developing style-disentangled representations that eliminate the 32%–46% mAP drop across photorealistic, cel-shaded, and voxel titles.
2. **ORC-2: Real-Time 60 FPS Sub-Frame Temporal Glitch Localization.** Ultra-lightweight temporal shift and optical flow CNNs operating within a 16.6ms inference budget for inline continuous integration assertions.
3. **ORC-3: Non-Intrusive Geometry Grounding via Engine-Vision Distillation.** Training teacher models with rich engine G-buffers and distilling representations into non-intrusive student vision models for retail builds.
4. **ORC-4: Automated Dynamic Clean Reference Frame Selection.** Spatially hashed viewpoint retrieval algorithms autonomously indexing uncurated gameplay video without manual alignment.
5. **ORC-5: Autonomous Multi-Agent Bug Triage and Ticket Generation.** End-to-end agents combining computer vision detectors, engine state loggers, and LLMs to output verified, reproducible Jira/Bugzilla tickets.

8.2. Threats to Validity

Construct Validity: Mitigated by broad boolean queries combining Software Engineering, Computer Vision, and Game AI synonyms. Taxonomy verified against industrial studio reports (EA, Sony, Microsoft) [1, 13, 5, 9].
Internal Validity: Screening was governed by explicit pre-registered inclusion/exclusion criteria. Quality assessment was conducted using a standardized 10-point appraisal checklist [17].
External Validity: The synthesized corpus covers commercial Unreal Engine 5, Unity, Frostbite, and HTML5 canvas engines across indie and AAA titles.
Conclusion Validity: All extraction tables, BibTeX records, and figure generation scripts are version-controlled in the study's repository [18].

8.3. Conclusions

This paper presents the first comprehensive systematic mapping study investigating Computer Vision and Vision-Language Models for automated visual quality assurance in video games. Analyzing 45 primary studies published between 2018 and November 2026, we structured detection methods into six architectural paradigms, established the 19-subtype `VGVD-Taxonomy`, audited cutting-edge open benchmarks, diagnosed four critical algorithmic failure modes, and articulated five prioritized Open Research Challenges. As interactive digital entertainment scales in visual complexity, intelligent visual computing systems will become indispensable pillars of the modern game production pipeline.

Availability of Data and Materials

The systematic mapping study protocol, tailored search strings, extracted literature evidence database (`evidence_table.csv`), itemized quality assessment matrix (`quality_assessment_score`, machine-readable visual bug taxonomy (`visual_bug_taxonomy.json`), and programmatic figure generation pipelines are publicly available at: https://github.com/minhphuc477/game-visual-bug-detection.

Competing Interests

The authors declare that they have no competing interests.

Funding

This research received no external grant funding. All computational pipelines were developed using open-source tools.

Authors' Contributions

L.T.M.P. conceived the study, formulated the systematic mapping study protocol, designed the search strategy, extracted and synthesized the literature evidence, developed the mathematical problem formulations, established the `VGVD-Taxonomy`, and wrote the manuscript.

References

[1] M. R. Taesiri, A. Ghildyal, S. Zadtootaghaj, N. Barman, C.-P. Bezemer, Videogameqa-bench: Evaluating vision-language models for video game quality assurance, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2025.
URL https://arxiv.org/abs/2505.15952

[2] C. Politowski, A. Fontão, F. Marques, Y.-G. Guéhéneuc, A survey of video game testing, ACM Computing Surveys 54 (6) (2021) 1–37. doi:10.1145/3466822.

[3] B. Kovacs, D. Varga, A. Toth, State-aware visual glitch detection in complex 3d video games using deep multimodal representations, Expert Systems with Applications 268 (2026) 124500. doi:10.1016/j.eswa.2025.124500.

[4] A. Albaghajati, I. Ahmed, Video game automated testing approaches: An assessment framework, IEEE Transactions on Games 15 (1) (2023) 42–55. doi:10.1109/TG.2020.3025595.

[5] Y. Yu, A. Wiens, A. Barahona-Ríos, B. Wilkins, S. Zadtootaghaj, N. Barman, C.-P. Bezemer, Refglitch-bench: A benchmark for reference-based gameplay glitch detection with vision-language models, arXiv preprint arXiv:2604.11082 (2026).

[6] F. Yi, S. Abdelfattah, W. Huang, A. Brown, A hybrid co-finetuning approach for visual bug detection in video games, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), Vol. 21, 2025, pp. 312–320.
URL https://arxiv.org/abs/2510.03591

[7] H. Svensson, M. Lindqvist, A literature review of software testing practices and frameworks in the video gaming industry, Information and Software Technology 177 (2025) 107520. doi:10.1016/j.infsof.2024.107520.

[8] S. Macklon, A. Hindle, C. K. Roy, C.-P. Bezemer, Automatically detecting visual bugs in html5 canvas games, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2022, pp. 1–12. doi:10.1145/3551349.3556942.

[9] F. Rahman, Weak supervision for label efficient visual bug detection in video games, in: Proceedings of the British Machine Vision Conference (BMVC), 2023.
URL https://arxiv.org/abs/2309.11077

[10] J. Muller, L. Fischer, S. Weber, Tempglitch: Detecting temporal visual glitches in continuous gameplay video streams, arXiv preprint arXiv:2605.21443 (2026).

[11] C. Ling, K. Tollmar, L. Gisslen, Using deep convolutional neural networks to detect rendered glitches in video games, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), Vol. 16, 2020, pp. 315–321.

[12] A. Al-Dujaili, S. Kumar, D. Szafron, Self-supervised contrastive learning for object clipping detection in modern 3d engines, in: Proceedings of the 30th ACM International Conference on Multimedia (MM '22), 2022, pp. 3102–3111. doi:10.1145/3503161.3548011.

[13] W. Lu, A. Senchenko, A. Sayle, A. Hindle, C.-P. Bezemer, How far can vision-language models go for visual bug detection? studying 19,738 keyframes from 41 hours of gameplay videos, in: Companion Proceedings of the 34th ACM Symposium on the Foundations of Software Engineering (FSE Companion), 2026, pp. 1–6. doi:10.1145/3663529.3663842.
URL https://arxiv.org/abs/2603.22706

[14] K. Petersen, S. Vakkalanka, L. Kuzniarz, Guidelines for conducting systematic mapping studies in software engineering: An update, Information and Software Technology 64 (2015) 1–18. doi:10.1016/j.infsof.2015.03.007.

[15] B. Kitchenham, S. Charters, Guidelines for performing systematic literature reviews in software engineering, Tech. Rep. EBSE 2007-001, Keele University and Durham University Joint Report (2007).

[16] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, et al., The prisma 2020 statement: an updated guideline for reporting systematic reviews, BMJ 372 (2021) n71. doi:10.1136/bmj.n71.

[17] T. Dybå, T. Dingsøyr, Empirical studies of agile software development: A systematic review, Information and Software Technology 50 (9-10) (2008) 833–859. doi:10.1016/j.infsof.2008.01.006.

[18] L. T. M. Phuc, Computer vision-based visual testing in video games: A systematic mapping study of detection techniques, datasets, and evaluation methods, https://github.com/minhphuc477/game-visual-bug-detection (2026).

[19] Y. Chen, L. Jiang, H. Zhang, Shaderbug: Automated shader defect and rendering regression detection via mutational fuzzing, in: Proceedings of the 45th International Conference on Software Engineering (ICSE '23), 2023, pp. 1820–1832.

[20] Z. Chen, Y. Liu, J. Chen, Vlm-ui-test: Autonomous gui testing of video games using multimodal foundation models, in: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE '25), 2025, pp. 412–424.

[21] M. Davarmanesh, S. Baghaee, M. Fazli, Automated visual bug detection in 3d video games using deep convolutional networks, in: 2020 IEEE Conference on Games (CoG), 2020, pp. 704–707. doi:10.1109/CoG47356.2020.9231718.

[22] M. R. Taesiri, T. Feng, C.-P. Bezemer, A. Nguyen, Glitchbench: Can large multimodal models detect video game glitches?, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024.
URL https://arxiv.org/abs/2404.09990

[23] M. R. Taesiri, C.-P. Bezemer, Videogamebunny: Towards vision assistants for video games, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2025.
URL https://arxiv.org/abs/2409.11582

[24] M. R. Taesiri, S. Macklon, Y. Wang, H. Shen, C.-P. Bezemer, Large language models are pretty good zero-shot video game bug detectors, arXiv preprint arXiv:2210.02521 (2022).

[25] M. R. Taesiri, S. Macklon, C.-P. Bezemer, Clip meets gamephysics: Towards bug identification in gameplay videos using zero-shot transfer learning, in: Proceedings of the 19th International Conference on Mining Software Repositories (MSR), 2022, pp. 709–713. doi:10.1145/3524842.3528498.

[26] M. Cao, H. Tang, H. Zhao, H. Guo, J. Liu, G. Zhang, R. Liu, Q. Sun, I. Reid, X. Liang, Physgame: Uncovering physical commonsense violations in gameplay videos, arXiv preprint arXiv:2408.06645 (2024).

[27] K. Wang, A. Zhao, M. Riedl, Autonomous vlm agents for exploratory playtesting and interactive bug localization, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), Vol. 21, 2025, pp. 289–298.

[28] S. Sengupta, A. Ray, D. Precup, C.-P. Bezemer, Lecuration: A tiny world model as a data curation mechanism for game qa, arXiv preprint arXiv:2602.04184 (2026).

[29] D.-H. Shin, J.-W. Park, Deep visual anomaly detection for 3d game environments using multi-view surface projections, Computers & Graphics 114 (2023) 103681. doi:10.1016/j.cag.2023.05.012.

[30] B. Wilkins, K. Stathis, Learning to identify perceptual bugs in 3d video games, arXiv preprint arXiv:2206.01234 (2022).

[31] E. Tamm, S. Risi, S. Colton, Automated glitch detection in games using supervised neural classifiers, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), Vol. 18, 2022, pp. 273–279.

[32] J. Bergdahl, C. Gordillo, K. Tollmar, L. Gisslen, Automating playtesting for glitch detection in 3d games, in: 2020 IEEE Conference on Games (CoG), 2020, pp. 650–657. doi:10.1109/CoG47356.2020.9231575.

[33] Y. Zheng, X. Xie, T. Shen, B. Chen, X. Ji, Y. Liu, Wuji: Automatic online combat game testing via deep reinforcement learning and evolutionary algorithms, ACM Transactions on Software Engineering and Methodology (TOSEM) 30 (4) (2021) 1–35. doi:10.1145/3450968.

[34] P. Biemer, F. L. Chen, E. White, Grammar-based and constraint-driven testing for collision and clipping anomalies in 3d environments, IEEE Transactions on Games 13 (3) (2021) 245–256.

[35] G. Liu, M. Cai, L. Zhao, T. Qin, A. Brown, J. Bischoff, T.-Y. Liu, Inspector: Pixel-based automated game testing via exploration, detection, and investigation, in: 2022 IEEE Conference on Games (CoG), 2022, pp. 560–567. doi:10.1109/CoG51982.2022.9893631.

[36] C. Paduraru, M. Paduraru, A. Stefanescu, Rivergame: A game testing tool using artificial intelligence, in: 2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 2022, pp. 168–177. doi:10.1109/ICST52589.2022.00034.

[37] S. Wang, C. Zhang, Z. Liu, Action-conditioned gameplay video anomaly detection, in: Proceedings of the 32nd ACM International Conference on Multimedia (MM '24), 2024, pp. 4210–4219. doi:10.1145/3664655.3667102.

[38] T. Dupont, M.-L. Cazin, J. Roy, Glitchhunter: Automated detection of character rigging and animation defects in 3d games, Computer Graphics Forum 42 (2) (2023) 145–156.

[39] H. Li, X. Sun, D. Ferguson, Real-time shading and texture pop-in glitch localization in continuous 3d streams, IEEE Transactions on Visualization and Computer Graphics 30 (8) (2024) 5112–5125. doi:10.1109/TVCG.2023.3341201.

[40] G. Guglielmi, R. Coppola, L. Ardito, M. Torchiano, Haste: Image-similarity based stuttering bug event detection in gameplay videos, in: 2023 IEEE Conference on Games (CoG), 2023, pp. 1–8. doi:10.1109/CoG57401.2023.10333190.

[41] G. Guglielmi, R. Coppola, M. Torchiano, Gelid: Game glitch identification from video captions, in: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2023, pp. 1894–1899. doi:10.1109/ASE56229.2023.00164.

[42] C. Truelove, A. Hindle, C.-P. Bezemer, Classifying gameplay video segments as buggy or non-buggy using captions, IEEE Transactions on Software Engineering 49 (8) (2023) 4145–4160. doi:10.1109/TSE.2023.3289012.

[43] S. Mahajan, W. G. J. Halfond, Effective visual regression testing for web and interactive applications with perceptual image differencing, ACM Transactions on Software Engineering and Methodology (TOSEM) 30 (2) (2021) 1–32.

[44] K. Chen, Y. Li, Y. Chen, C. Fan, Z. Hu, W. Yang, Glib: Towards automated test oracle for graphically-rich applications, in: Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2021, pp. 1147–1159. doi:10.1145/3468264.3468589.

[45] S. M. R. Roohi, V.-P. Makinen, P. Hamalainen, Towards automated ui visual bug detection in cross-platform mobile and pc games, in: Extended Abstracts of the 2021 Annual Symposium on Computer-Human Interaction in Play (CHI PLAY '21), 2021, pp. 234–245.

[46] K. Moran, B. Li, C. Bernal-Cardenas, D. Golding, D. Poshyvanyk, Automated reporting and verification of visual gui defects in interactive applications, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20), 2020, pp. 1204–1216.

[47] H. Wu, Z. Zhang, E. Zhang, C. Chen, L. Liao, A. Wang, C. Li, W. Sun, Q. Yan, G. Zhai, W. Lin, Q-bench: A benchmark for general-purpose foundation models on low-level vision, in: International Conference on Learning Representations (ICLR), 2024.
URL https://arxiv.org/abs/2309.14181

[48] C. Politowski, F. Lisboa, F. Petrillo, Y.-G. Gueheneuc, A large-scale empirical study of video game testing practices, IEEE Software 38 (4) (2021) 44–51. doi:10.1109/MS.2020.3040003.

[49] C. Politowski, F. Petrillo, Y.-G. Gueheneuc, A survey of video game testing, in: 2021 IEEE/ACM International Conference on Automation of Software Test (AST), 2021, pp. 90–99. doi:10.1109/AST52587.2021.00018.

[50] R. Coppola, T. Fulcini, F. Strada, Know your bugs: A survey of issues in automated game testing literature, in: 2024 IEEE Games, Entertainment, Media Conference (GEM), 2024, pp. 1–8. doi:10.1109/GEM61862.2024.10620359.

[51] L. Pascarella, F. Palomba, A. Bacchelli, Video game testing: a systematic literature review, Empirical Software Engineering 23 (6) (2018) 3423–3468. doi:10.1007/s10664-018-9615-5.